LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

15

Linux for Docker and Kubernetes

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Containers vs Virtual Machines
03 Docker Architecture
04 Namespaces and PID 1
05 Control Groups (cgroups)
06 Image Layers and OverlayFS
07 Volumes and Bind Mounts
08 Container Networking
09 Container Security
10 Container Runtimes and OCI
11 Linux Behind Kubernetes
12 Requests and Limits
13 Troubleshooting Scenarios
14 Command Cheat Sheet
15 Best Practices
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand the Linux kernel features that make containers work: namespaces, cgroups, OverlayFS, network namespaces and capabilities. Learn how Docker, containerd, runc and Kubernetes use them, and how to troubleshoot container and pod problems by looking at both the container and the underlying Linux host.

1 Introduction

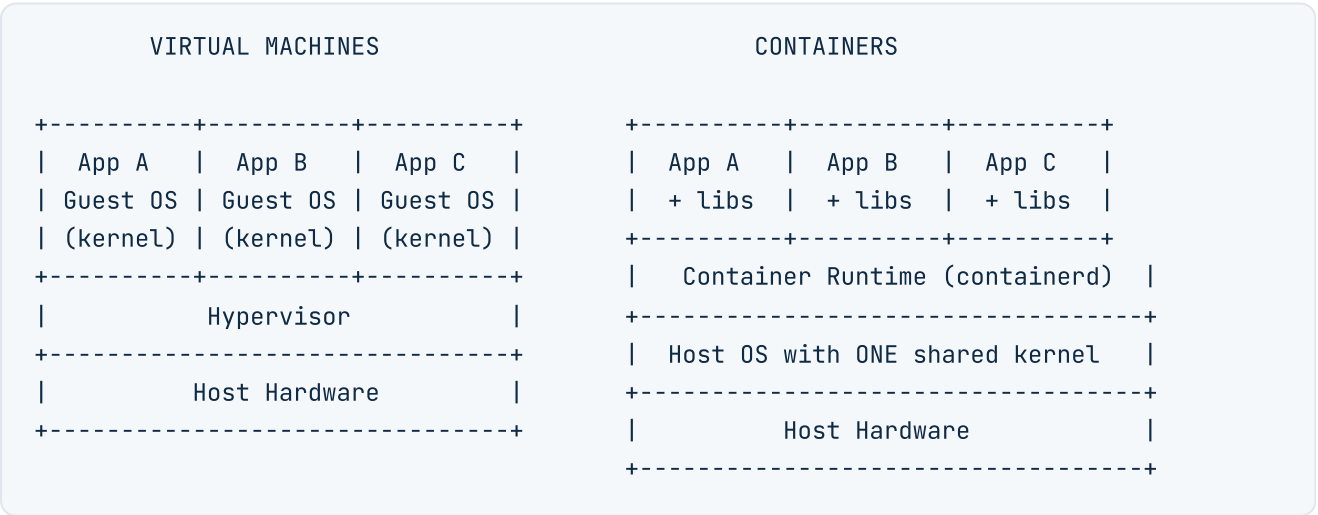
Docker packages applications and their dependencies into containers. Kubernetes runs and manages those containers across many servers. Both depend on Linux kernel features.

Knowing the Linux concepts behind containers helps you solve problems that Docker or Kubernetes commands alone cannot explain, such as:

- Why does a container use more memory than expected?
- Why can't a container reach the network?
- Why does a container get `Permission denied` even as root?
- Why does a container stop when its main process exits?
- Why is a Kubernetes pod `OOMKilled`?
- How do containers share the host kernel?

2 Containers vs Virtual Machines

A **virtual machine** runs a full guest operating system with its own kernel on virtualized hardware. A **container** is an isolated group of processes that shares the host's kernel but has its own view of processes, filesystem and network.



Feature	Virtual Machine	Container
Kernel	Each VM has its own	Shares the host kernel
Startup	Minutes (boots an OS)	Seconds or less (starts a process)
Resource usage	Higher	Lower
Isolation	Hardware virtualization (stronger)	Kernel features: namespaces and cgroups
Packaging	VM image	Container image
Tools	KVM, VMware, Hyper-V	Docker, containerd, Podman

IMPORTANT

Because containers share the host kernel, a kernel vulnerability or misconfiguration can affect every container on the host. A container also cannot run a different kernel: a Linux container needs a Linux kernel.

3 Docker Architecture

Docker uses a client-server design. The CLI talks to the Docker daemon, which hands lower-level work to containerd and runc.

```
docker CLI  (docker build / pull / run)
     |
     |  Docker API (unix socket /var/run/docker.sock)
     v
dockerd      → images, networks, volumes, build
     |
     v
containerd   → container lifecycle (start, stop, pull)
     |
     v
runc         → creates the container using kernel features, then exits
     |
     v
Linux kernel → namespaces | cgroups | OverlayFS | capabilities
```

Component	Role
Docker client	CLI used to send commands (`docker ps` , `docker run`)
Docker daemon (`dockerd`)	Manages images, containers, networks and volumes
containerd	Manages the container lifecycle; also used directly by Kubernetes
runc	Low-level OCI runtime that actually creates the container
Registry	Stores images, such as Docker Hub or Amazon ECR

```
sudo systemctl status docker
docker version
docker info
```

SECURITY WARNING

Membership in the `docker` group is effectively **root access** on the host. A member can run `docker run -v /:/host ...` and read or modify any host file. Grant it only to trusted administrators.

4 Linux Namespaces – Container Isolation

Namespaces give a process its own separate view of selected system resources. Container runtimes put each container in its own set of namespaces.

Namespace	Isolates
PID	Process IDs: the container has its own process tree, starting at PID 1
NET	Network interfaces, IP addresses, routes, firewall rules and sockets
MNT	Mount points: the container sees its own filesystem
UTS	Hostname and domain name
IPC	Inter-process communication resources (shared memory, queues)
USER	User and group IDs: root in the container can map to an unprivileged host user

A process inside a container may appear as PID 1, while the same process has a different PID on the host.

```
# All namespaces on the host
sudo lsns

# Find a container's main process PID on the host
docker inspect -f '{{.State.Pid}}' web

# See that process's namespaces
sudo ls -l /proc/<PID>/ns/

# Run a host tool inside the container's network namespace
sudo nsenter -t <PID> -n ip addr
```

DEVOPS TIP

Minimal images often have no `ip`, `ping`, `ss` or `curl`. With `nsenter`, you can use the host's tools inside a container's namespace without installing anything in the container.

PID 1 Inside a Container

Chapter 14 explained PID 1 on a normal host. Inside a container, PID 1 is usually your application, and that changes two things:

- **Signals:** the kernel does not apply default signal actions to PID 1. If the application does not handle SIGTERM itself, `docker stop` waits 10 seconds and then sends SIGKILL.
- **Zombies:** PID 1 must reap orphaned child processes. Most applications do not, so zombies can build up.

INTERVIEW TRAP

Using the shell form `CMD python app.py` makes `/bin/sh` PID 1, and the shell does not forward SIGTERM to your app. Use the exec form `CMD ["python", "app.py"]`, and use `docker run --init` (a tiny init such as tini) when the app spawns child processes.

5 Control Groups (cgroups) – Resource Limits

Namespaces control what a container can **see**. Cgroups control how much it can **use**: CPU, memory, number of processes and block I/O. Without limits, one container can starve every other workload on the host.

```
# Memory limit of 512 MB
docker run -d --name web --memory=512m nginx

# Limit to 1.5 CPUs and at most 200 processes
docker run -d --name api --cpus=1.5 --pids-limit=200 myapp:1.0

# Live usage
docker stats

# Configured limits (bytes; 0 = no limit)
docker inspect -f '{{.HostConfig.Memory}} {{.HostConfig.NanoCpus}}' web
```

cgroups v1 vs v2

cgroups v1 uses separate hierarchies per resource. cgroups v2 uses one unified hierarchy and is the default on modern distributions (Ubuntu 22.04+, RHEL 9+).

```
# cgroup2fs = v2, tmpfs = v1
stat -fc %T /sys/fs/cgroup/
```

> **IMPORTANT**
>
> A container that exceeds its memory limit is killed by the kernel's OOM killer **even if the host still has free RAM**. The limit, not total host memory, triggers the kill.

6 Image Layers and OverlayFS

A container image is built from layers. Each instruction that changes files in a Dockerfile creates a new read-only layer.

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y nginx && rm -rf /var/lib/apt/lists/*
COPY index.html /var/www/html/index.html
CMD ["nginx", "-g", "daemon off;"]
```

OverlayFS is a union filesystem that stacks these layers into one view:

```
+-----------------------------+
|  Merged view (what the      |
|  container sees as /)       |
+-----------------------------+
|  Upper: writable container  |  ← changes made by the running container
|  layer                      |
+-----------------------------+
|  Lower: read-only image     |  ← shared by every container from this image
|  layers                     |
+-----------------------------+
```

Many containers can share the same image layers on disk. When a container modifies a file from a lower layer, OverlayFS copies it into the writable layer first (copy-up).

```
docker image ls
docker history nginx       # layers of an image
docker system df           # disk used by images, containers and volumes
```

> **INTERVIEW LINE**
>
> **Images are immutable templates; a running container adds a thin writable layer on top.**
>
> Data in that writable layer is deleted when the container is removed. Persistent data belongs in volumes or bind mounts.

7 Volumes and Bind Mounts

Containers are replaced during every deployment, so important data must live outside the writable layer.

Docker Volumes

```
docker volume create app-data
docker volume ls
docker volume inspect app-data

docker run -d --name app -v app-data:/app/data myapp:1.0
```

Docker stores volume data on the host, by default under `/var/lib/docker/volumes/`.

Bind Mounts

```
# Map a host directory into the container, read-only
docker run -d --name web -v /opt/website:/usr/share/nginx/html:ro nginx
```

Feature	Volume	Bind Mount
Managed by	Docker	You (any host path)
Host location	Docker-managed directory	Explicitly specified path
Portability	Better	Depends on the host's directory layout
Common use	Databases and persistent app data	Configuration files and development code

> **IMPORTANT**
>
> Persistent storage is not a backup. Volumes can be deleted or corrupted, so back them up like any other data.

8 Container Networking

Each container normally gets its own network namespace. Docker connects it to the outside world using virtual Ethernet (veth) pairs and Linux bridges.

Network	Behavior
bridge	Default for standalone containers; private IPs behind NAT on the `docker0` bridge
host	Shares the host's network namespace; no isolation and no `-p` needed
none	Only a loopback interface
overlay	Connects containers across multiple hosts (Swarm)
macvlan	Gives a container its own MAC address on the physical network

User-Defined Bridge Networks

```
docker network create app-network

docker run -d --name web  --network app-network nginx
docker run -d --name test --network app-network busybox sleep 3600

# Containers on the same user-defined network resolve each other by name
docker exec test ping -c 2 web
```

> **INTERVIEW TRAP**
>
> Name resolution between containers works on **user-defined** networks through Docker's embedded DNS. On the default `bridge` network, containers can reach each other only by IP address, not by name.

Publishing Ports

```
# Host port 8080 forwards to container port 80
docker run -d --name web -p 8080:80 nginx

docker port web
docker network inspect app-network
```

Docker publishes ports by adding its own iptables/nftables NAT rules. On Ubuntu, a published port can be reachable even if UFW blocks it, because Docker's rules are evaluated first.

9 Container Security

Linux Capabilities

Capabilities split root's power into smaller privileges. Root inside a container receives only a restricted default set, not every capability.

Capability	Allows
`CAP_NET_BIND_SERVICE`	Binding to ports below 1024
`CAP_NET_ADMIN`	Network configuration (interfaces, routes, firewall)
`CAP_CHOWN`	Changing file ownership
`CAP_DAC_OVERRIDE`	Bypassing file permission checks
`CAP_SYS_ADMIN`	A very broad set of admin operations (avoid granting it)

```
# Drop everything, add back only what the app needs
docker run -d --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp:1.0

# Run as a non-root user
docker run -d --user 1000:1000 myapp:1.0

# Read-only root filesystem, with writable tmpfs where the app needs it
docker run -d --read-only --tmpfs /tmp myapp:1.0
```

These options must match the application. For example, the official `nginx` image needs extra capabilities (`CHOWN` , `SETUID` , `SETGID`) and writable `/var/cache/nginx` and `/var/run` ; the `nginxinc/nginx-unprivileged` image runs as non-root on port 8080.

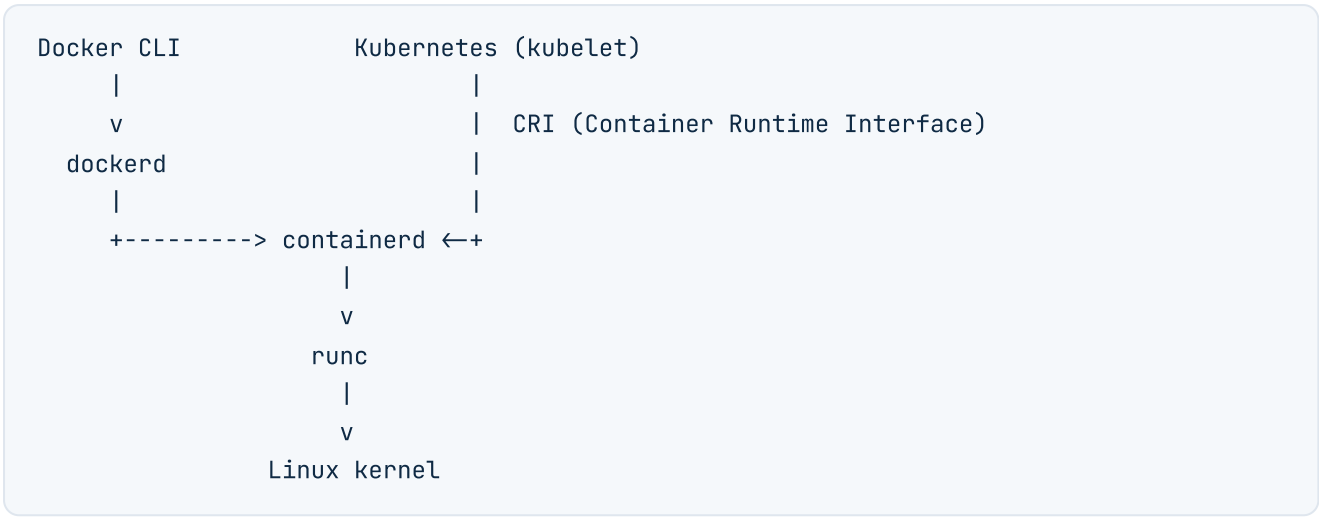
> **WARNING**
>
> Avoid `docker run --privileged` . It gives the container all capabilities and access to host devices, which largely removes isolation. Use it only when there is no alternative.

Other Security Controls

- **Seccomp:** blocks dangerous system calls (Docker applies a default profile).
- **AppArmor / SELinux:** mandatory access control for container processes (see Chapter 9).
- **User namespaces / rootless containers:** root in the container maps to an unprivileged host user.
- **Trusted, scanned images:** keep base images small and updated.

10 Container Runtimes and OCI

The **Open Container Initiative (OCI)** defines two standards: the **image specification** (how images are packaged) and the **runtime specification** (how a runtime runs a container). Any OCI image runs on any OCI runtime.



INTERVIEW NOTE

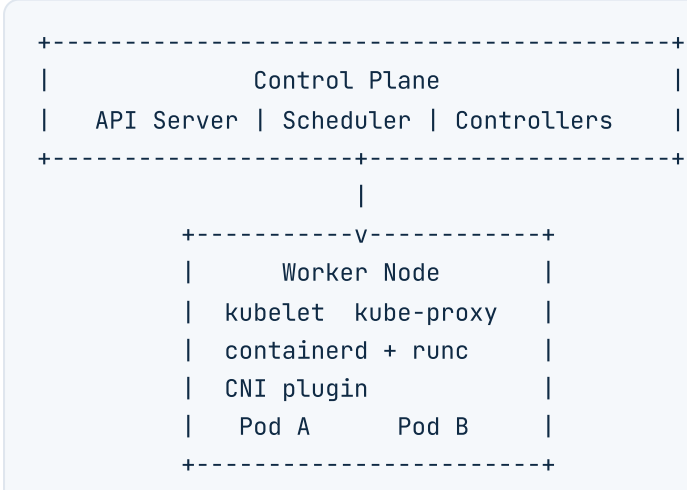
Since Kubernetes 1.24, Kubernetes no longer uses Docker Engine as its runtime (dockershim was removed). Nodes use containerd or CRI-O directly through CRI. Images built with Docker still run normally, because they are OCI images.

```
systemctl status containerd
containerd --version
runc --version

# On Kubernetes nodes: inspect the CRI runtime directly
sudo crictl ps
sudo crictl pods
sudo crictl info
```

11 Linux Concepts Behind Kubernetes

Kubernetes schedules containers onto worker nodes. On each node, Linux does the actual work.



Linux feature	Kubernetes usage
Namespaces	Pod isolation; containers in one pod share a network namespace
cgroups	Enforcing CPU and memory requests and limits
Container runtime	Starting and stopping containers
Filesystems	Image storage and volumes
Networking (veth, routes, iptables)	Pod networking via CNI and Services via kube-proxy
Kernel security	Capabilities, seccomp and security contexts

Component	Role
kubelet	Node agent that makes sure the pod's containers are running
Container runtime	Runs containers, commonly containerd
kube-proxy	Implements Service networking rules on each node (some CNIs replace it)
CNI plugin	Gives pods their network interfaces and IP addresses

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl describe node <node-name>

# On the node itself
systemctl status kubelet
journalctl -u kubelet -n 100
```

DEVOPS POINT

Kubernetes does not replace Linux; it depends on Linux.

When pods misbehave, the cause is often on the node: disk pressure, memory pressure, a failed kubelet or a runtime problem.

12 Kubernetes Requests and Limits

Requests are what the scheduler reserves when placing a pod on a node. **Limits** are the maximum the container may use, enforced by cgroups.

```
resources:
  requests:
    memory: "128Mi"
    cpu: "250m"      # 0.25 CPU
  limits:
    memory: "256Mi"
    cpu: "500m"      # 0.5 CPU
```

Resource	When the limit is reached
Memory	The container is **killed** (`OOMKilled` , exit code 137) and restarted
CPU	The container is **throttled**: it keeps running but slower

```
kubectl get pods
kubectl describe pod app-pod      # look for "Reason: OOMKilled"
kubectl top pod                   # needs metrics-server
```

> INTERVIEW LINE
>
> **Memory limits kill; CPU limits throttle.**
>
> CPU throttling often shows up as unexplained latency rather than an error, so it is easy to miss.

13 Troubleshooting Containers with Linux

Container Exit Codes

Exit code	Meaning
0	Main process finished successfully
1	Application error
137	Killed by SIGKILL (128 + 9): usually OOM, or a forced stop
143	Terminated by SIGTERM (128 + 15): normal graceful stop

Scenario 1 Container uses too much memory or is killed

```
docker stats
docker inspect -f '{{.State.OOMKilled}} {{.State.ExitCode}}' app
free -h
journalctl -k | grep -i "out of memory"
```

Decide whether the cause is the application's memory use, a limit that is too low, or memory pressure on the whole host.

Scenario 2 Container cannot reach another container

```
docker network inspect app-network     # are both attached?
docker exec test ping -c 2 web         # name resolution and reachability

# Is the target listening, and on which address?
sudo nsenter -t $(docker inspect -f '{{.State.Pid}}' web) -n ss -lntp
```

Check that both containers share a user-defined network, the name resolves, and the application listens on `0.0.0.0`, not `127.0.0.1`.

Scenario 3 Permission denied inside a container

```
docker exec app id                 # which UID/GID is the process?
docker exec app ls -ln /app/data   # numeric ownership inside
ls -ln /opt/app/data               # numeric ownership on the host (bind mount)
```

With bind mounts, ownership is matched by **numeric UID**, not by username. Also check read-only mounts, dropped capabilities and SELinux: on RHEL-based hosts, add `:z` or `:Z` to the bind mount so it gets the right label.

Scenario 4 Container exits immediately

```
docker ps -a
docker logs app
docker inspect -f '{{.State.ExitCode}}' app
```

A container stops when its main process exits. Make sure the application runs in the foreground, for example `nginx -g 'daemon off;'`, instead of starting a background daemon and exiting.

Scenario 5 Pod is in CrashLoopBackOff

```
kubectl describe pod <pod>        # events, last state, exit code
kubectl logs <pod>
kubectl logs <pod> --previous     # logs from the crashed instance
```

Common causes: startup errors, missing configuration or secrets, wrong environment variables, unavailable dependencies, permission problems, memory limits and failing liveness probes.

Scenario 6 Pod is stuck in Pending

```
kubectl describe pod <pod>        # read the Events section
kubectl describe nodes | grep -A 5 "Allocated resources"
```

Common causes: not enough CPU or memory to satisfy requests, node selectors or affinity rules, taints without tolerations, or an unbound PersistentVolumeClaim.

14 Command Cheat Sheet

Requirement	Command
Docker service status	`systemctl status docker`
Running / all containers	`docker ps` / `docker ps -a`
Container details	`docker inspect NAME`
Resource usage	`docker stats`
Disk usage	`docker system df`
Image layers	`docker history IMAGE`
Networks / volumes	`docker network ls` / `docker volume ls`
Container's host PID	`docker inspect -f '{{.State.Pid}}' NAME`
Namespaces	`lsns`, `ls -l /proc/PID/ns/`
Use host tools in a container	`nsenter -t PID -n ip addr`
cgroup version	`stat -fc %T /sys/fs/cgroup/`
CRI containers / pods	`crictl ps` / `crictl pods`
Nodes and pods	`kubectl get nodes -o wide`, `kubectl get pods -A`
Pod events	`kubectl describe pod NAME`
Previous container logs	`kubectl logs NAME --previous`
Kubelet logs	`journalctl -u kubelet`

15 Best Practices

1. **Set CPU and memory limits** for every container.
2. **Run as non-root** and drop unneeded capabilities.
3. **Avoid privileged containers** and restrict the `docker` group.
4. **Use exec-form CMD** so the application receives signals, and `--init` when it spawns children.
5. **Keep persistent data in volumes**, and back them up.
6. **Use user-defined networks** for name-based container communication.
7. **Use small, trusted, scanned images** and update them regularly.

8. **Investigate OOM events** at both container and host level.
9. **Check the node** (kubelet, disk, memory) when pods misbehave.
10. **Collect logs and events** before restarting workloads.

16 Common Interview Questions

Q1. What is the difference between a container and a virtual machine?

A VM runs its own guest OS and kernel on a hypervisor. A container is an isolated process group that shares the host kernel, isolated with namespaces and limited with cgroups.

Q2. What is the difference between namespaces and cgroups?

Namespaces control what a container can see (isolation). Cgroups control how much it can use (resource limits).

Q3. What is OverlayFS?

A union filesystem that stacks read-only image layers with a writable container layer into one merged view.

Q4. Why should data not be stored only inside a container?

The writable layer is deleted with the container. Use volumes or bind mounts for persistent data.

Q5. What is the difference between a volume and a bind mount?

A volume is managed by Docker in its own directory. A bind mount maps a specific host path into the container.

Q6. What is the role of containerd and runc?

containerd manages the container lifecycle. runc is the low-level OCI runtime that creates the container using kernel features.

Q7. What is OCI?

The Open Container Initiative defines standard image and runtime specifications, so images work across different runtimes.

Q8. Does Kubernetes still use Docker?

Not as its runtime. Since Kubernetes 1.24, nodes use containerd or CRI-O through CRI. Docker-built images still run because they follow the OCI standard.

Q9. What happens when a container exceeds its memory limit versus its CPU limit?

Exceeding the memory limit gets the container OOM-killed (exit code 137). Reaching the CPU limit causes throttling, which slows it down.

Q10. What does exit code 137 mean?

The process was killed by SIGKILL (128 + 9), most often by the OOM killer or a forced stop.

Q11. Why does `docker stop` sometimes take 10 seconds?

The PID 1 process did not handle SIGTERM, so Docker waited for the timeout and then sent SIGKILL. Use exec-form CMD and handle SIGTERM in the app, or use `--init`.

Q12. Why does a container stop immediately after starting?

Its main process exited. The application must run in the foreground as the container's main process.

Q13. Why can't containers on the default bridge network reach each other by name?

Docker's embedded DNS works only on user-defined networks. On the default bridge, use IP addresses or create a user-defined network.

Q14. Why is membership in the docker group dangerous?

A member can start a container that mounts the host's root filesystem, which is equivalent to root access on the host.

Q15. Why might a container get Permission denied even when running as root?

Dropped capabilities, user namespace mapping, read-only mounts, SELinux or AppArmor restrictions, or host file ownership on a bind mount.

Q16. What do kubelet and the CNI plugin do?

kubelet is the node agent that keeps the pod's containers running. The CNI plugin gives pods their network interfaces and IP addresses.

Q17. What is CrashLoopBackOff, and how do you investigate it?

The container keeps exiting and Kubernetes is delaying restarts. Check `kubectl describe pod` for events and exit codes, and `kubectl logs --previous` for the crashed instance.

Q18. Why might a pod stay in Pending?

Not enough resources for its requests, node selectors or affinity rules, taints without tolerations, or an unbound PersistentVolumeClaim.

Q19. How do you troubleshoot a container's network when the image has no network tools?

Find the container's host PID with `docker inspect`, then run host tools in its namespace with `nsenter -t PID -n`.

17 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ How containers differ from virtual machines.
- ✓ Docker architecture: dockerd, containerd and runc.
- ✓ How namespaces isolate containers, and how PID 1 behaves inside them.
- ✓ How cgroups enforce CPU, memory and process limits.
- ✓ Image layers and OverlayFS.
- ✓ Volumes vs bind mounts.
- ✓ Container networking, user-defined networks and port publishing.
- ✓ Capabilities, non-root containers and the risks of privileged mode.
- ✓ OCI, CRI and why Kubernetes no longer uses Docker as its runtime.
- ✓ How Kubernetes nodes use Linux features.
- ✓ Requests, limits, OOMKilled and CPU throttling.
- ✓ Troubleshooting containers and pods from the Linux host.

PRACTICAL EXERCISES

Use a Linux machine with Docker, and a test cluster such as Minikube or Kind for the Kubernetes tasks.

1. Start an Nginx container and compare its PID inside the container and on the host.
2. List the container's namespaces using `/proc/PID/ns/` and `lsns`.
3. Run `ip addr` inside the container's network namespace using `nsenter`.
4. Start a container with `--memory` and `--cpus` limits and watch it with `docker stats`.
5. Check whether your host uses cgroups v1 or v2.
6. Inspect an image's layers with `docker history`.
7. Create a volume, write data to it, remove the container, and confirm the data survives.
8. Create a read-only bind mount and test writing to it.
9. Create a user-defined network and test name resolution between two containers.
10. Run a container as a non-root user and check it with `id`.
11. Compare `docker stop` time for a shell-form and an exec-form CMD.
12. Deploy a pod with requests and limits in a test cluster.
13. Trigger an `OOMKilled` event in a test pod and investigate it.
14. Investigate a pod in `CrashLoopBackOff` using `--previous` logs.
15. Investigate a pod stuck in `Pending`.
16. Identify the container runtime on a Kubernetes node using `crictl info`.

INTERVIEW TARGET

Be able to explain how namespaces, cgroups, OverlayFS and container runtimes work, and troubleshoot common Docker and Kubernetes problems using both container-level and host-level information.

NEXT CHAPTER

Chapter 16: Real-World Linux Production Scenarios and Troubleshooting

You will learn how to investigate common production incidents step by step, from symptoms to root cause, and explain your troubleshooting decisions confidently in interviews.

END OF CHAPTER 15